Case Study 1: Student Grade Calculator

1. Problem Statement

A teacher wants a simple program to calculate a student's total, average, and grade from marks in three subjects.

2. Objectives

• Accept marks for three subjects.

• Calculate total and average marks.

• Assign a grade using simple conditions.

3. Assumptions

• Each subject is marked out of 100.

• Marks entered are between 0 and 100.

• Average 90 or above = A; 75–89.99 = B; 60–74.99 = C; otherwise = D.

4. Algorithm

1. Start.

2. Input marks for three subjects.

3. Calculate total and average.

4. Use if-elif-else to assign a grade.

5. Display total, average, and grade.

6. Stop.

5. Flowchart

```
START
  |
Input 3 subject marks
  |
Calculate total and average
  |
Assign grade using conditions
  |
Display result
  |
 STOP
```

6. Pseudocode

```
START
  INPUT mark1, mark2, mark3
  total = mark1 + mark2 + mark3
  average = total / 3
  IF average >= 90: grade = 'A'
  ELSE IF average >= 75: grade = 'B'
  ELSE IF average >= 60: grade = 'C'
  ELSE: grade = 'D'
  DISPLAY total, average, grade
STOP
```

7. Python Source Code

```
m1 = float(input("Enter marks for Subject 1: "))
m2 = float(input("Enter marks for Subject 2: "))
m3 = float(input("Enter marks for Subject 3: "))

total = m1 + m2 + m3
average = total / 3

if average >= 90:
    grade = "A"
elif average >= 75:
    grade = "B"
elif average >= 60:
    grade = "C"
else:
    grade = "D"

print("Total:", total)
print("Average:", round(average, 2))
print("Grade:", grade)
```

8. Test Cases

Subject 1	Subject 2	Subject 3	Total	Average	Grade
80	75	85	240	80.00	B
95	92	90	277	92.33	A
40	55	50	145	48.33	D

9. Sample Output

```
Enter marks for Subject 1: 80
Enter marks for Subject 2: 75
Enter marks for Subject 3: 85
Total: 240.0
Average: 80.0
Grade: B
```

10. Conclusion

The program practices variables, arithmetic operators, input/output, and conditional statements to summarize academic performance.

Case Study 2: Simple Electricity Bill

1. Problem Statement

A household wants to estimate its electricity bill from the number of units consumed.

2. Objectives

• Read electricity units.

• Calculate the charge using a simple slab rule.

• Display the final bill.

3. Assumptions

• First 100 units cost ■2 per unit.

• Units above 100 cost ■3 per unit.

• A fixed charge of ■50 is added.

• Consumption is zero or more.

4. Algorithm

1. Start.

2. Input units consumed.

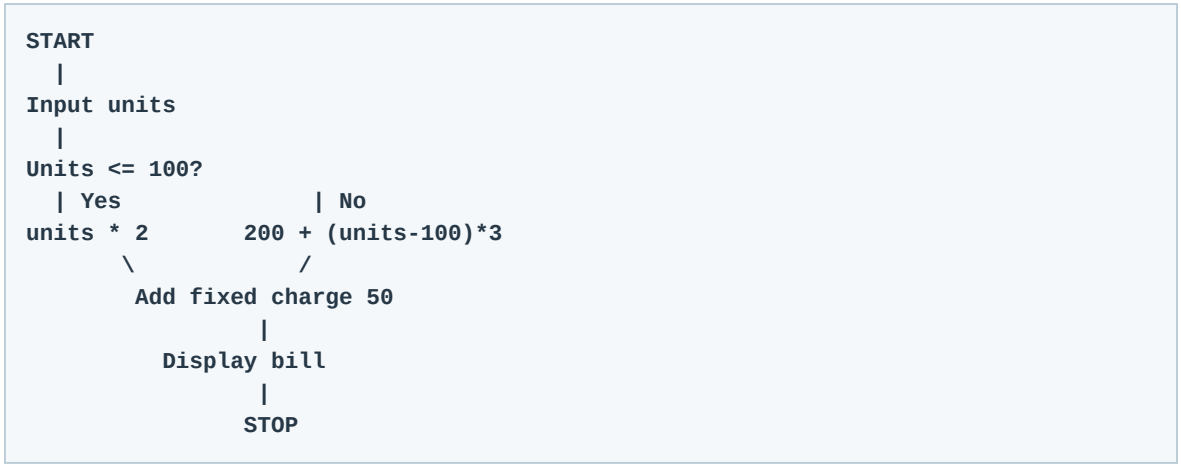
3. If units are at most 100, calculate units × 2.

4. Otherwise, calculate 100 × 2 plus remaining units × 3.

5. Add fixed charge of ■50.

6. Display bill and stop.

5. Flowchart



6. Pseudocode

```
START
  INPUT units
  IF units <= 100
      charge = units * 2
  ELSE
      charge = 100 * 2 + (units - 100) * 3
  bill = charge + 50
  DISPLAY bill
STOP
```

7. Python Source Code

```
units = int(input("Enter electricity units: "))

if units <= 100:
    charge = units * 2
else:
    charge = 100 * 2 + (units - 100) * 3

bill = charge + 50
print("Energy charge: Rs.", charge)
print("Fixed charge: Rs. 50")
print("Total bill: Rs.", bill)
```

8. Test Cases

Units	Energy charge	Total bill
80	160	210
100	200	250
150	350	400

9. Sample Output

```
Enter electricity units: 150
Energy charge: Rs. 350
Fixed charge: Rs. 50
Total bill: Rs. 400
```

10. Conclusion

The program uses if-else statements and arithmetic to apply a basic electricity tariff.

Case Study 3: Shopping Discount Calculator

1. Problem Statement

A shop gives a discount based on the total purchase amount. The program calculates the discount and final amount.

2. Objectives

• Input the purchase amount.

• Apply the correct discount rate.

• Display the discount and amount to pay.

3. Assumptions

• Purchase below ■1000 receives no discount.

• ■1000 to ■2999 receives 5% discount.

• ■3000 or more receives 10% discount.

• The purchase amount cannot be negative.

4. Algorithm

1. Start.

2. Input purchase amount.

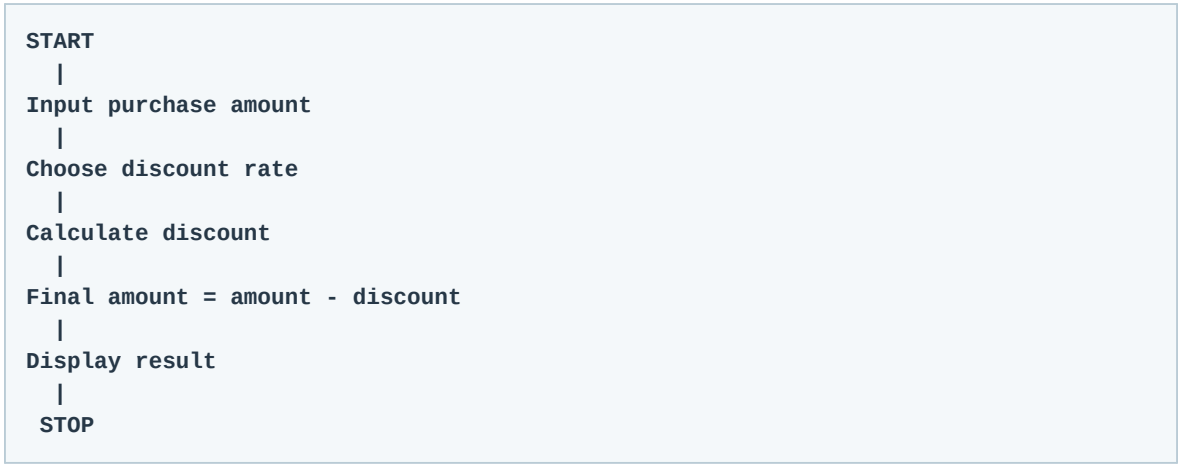
3. Select discount rate using conditions.

4. Calculate discount and final amount.

5. Display both values.

6. Stop.

5. Flowchart



6. Pseudocode

```
START
  INPUT amount
  IF amount >= 3000: rate = 0.10
  ELSE IF amount >= 1000: rate = 0.05
  ELSE: rate = 0
  discount = amount * rate
  payable = amount - discount
  DISPLAY discount, payable
STOP
```

7. Python Source Code

```
amount = float(input("Enter purchase amount: "))

if amount >= 3000:
    rate = 0.10
elif amount >= 1000:
    rate = 0.05
else:
    rate = 0

discount = amount * rate
payable = amount - discount

print("Discount: Rs.", round(discount, 2))
print("Amount to pay: Rs.", round(payable, 2))
```

8. Test Cases

Purchase amount	Discount	Payable
800	0	800
2000	100	1900
4000	400	3600

9. Sample Output

```
Enter purchase amount: 2000
Discount: Rs. 100.0
Amount to pay: Rs. 1900.0
```

10. Conclusion

This case study demonstrates decision-making with if-elif-else and percentage calculations.

Case Study 4: Even or Odd Number Checker

1. Problem Statement

A student wants to determine whether a whole number is even or odd.

2. Objectives

• Read an integer.

• Use the remainder operator.

• Print whether the number is even or odd.

3. Assumptions

• The input is a whole number.

• A number divisible by 2 is even.

4. Algorithm

1. Start.

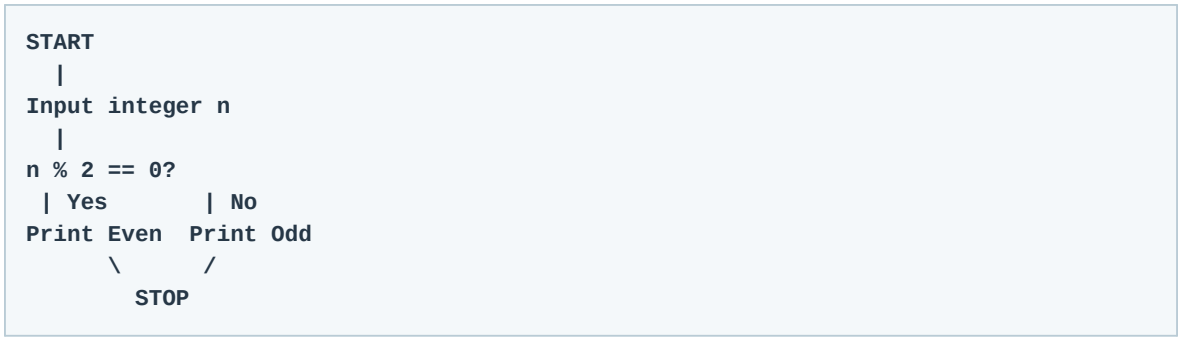
2. Input an integer.

3. Find the remainder after division by 2.

4. If remainder is zero, display Even; otherwise display Odd.

5. Stop.

5. Flowchart



6. Pseudocode

```
START
  INPUT n
  IF n % 2 == 0
      DISPLAY 'Even'
  ELSE
      DISPLAY 'Odd'
STOP
```

7. Python Source Code

```
n = int(input("Enter a whole number: "))

if n % 2 == 0:
    print(n, "is even")
else:
    print(n, "is odd")
```

8. Test Cases

Input number	Expected result
12	Even
7	Odd
0	Even

9. Sample Output

```
Enter a whole number: 7
7 is odd
```

10. Conclusion

The program introduces the modulus operator and a simple if-else condition.

Case Study 5: Weekly Temperature Analysis

1. Problem Statement

A weather record stores temperatures for seven days. The program calculates the average and identifies the highest temperature.

2. Objectives

• Store seven temperatures in a list.

• Calculate the average temperature.

• Find and display the maximum temperature.

3. Assumptions

• Exactly seven numeric temperatures are entered.

• Temperatures may be positive or negative.

4. Algorithm

1. Start.

2. Create an empty list.

3. Repeat seven times to input a temperature and append it.

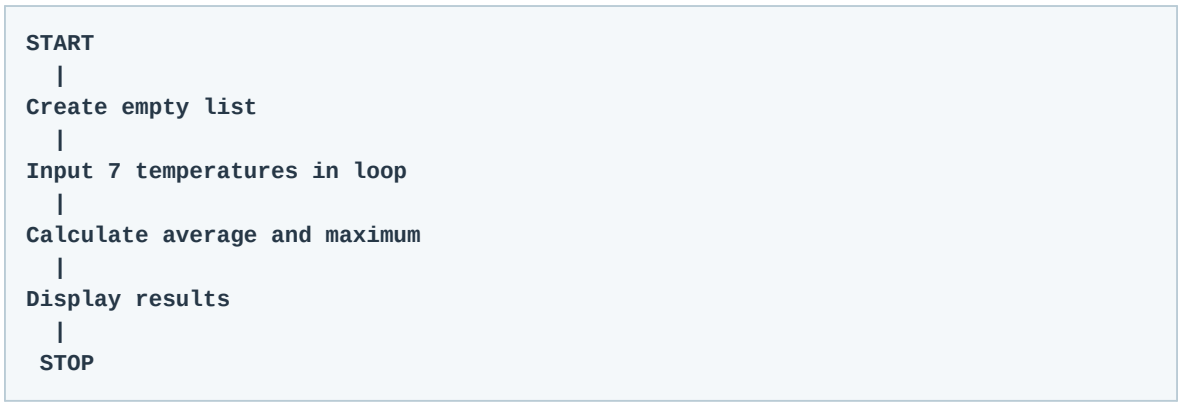
4. Calculate the average using sum divided by seven.

5. Find the highest value using max.

6. Display average and highest temperature.

7. Stop.

5. Flowchart



6. Pseudocode

```
START
  temperatures = empty list
  REPEAT 7 times
      INPUT temperature
      APPEND temperature
  average = SUM(temperatures) / 7
  highest = MAX(temperatures)
  DISPLAY average, highest
STOP
```

7. Python Source Code

```
temperatures = []

for day in range(1, 8):
    temp = float(input("Enter temperature for day " + str(day) + ": "))
    temperatures.append(temp)

average = sum(temperatures) / len(temperatures)
highest = max(temperatures)

print("Temperatures:", temperatures)
print("Average temperature:", round(average, 2))
print("Highest temperature:", highest)
```

8. Test Cases

Daily temperatures	Average	Highest
30,32,31,29,33,34,30	31.29	34
20,21,19,22,18,20,21	20.14	22

9. Sample Output

```
Temperatures: [30, 32, 31, 29, 33, 34, 30]
Average temperature: 31.29
Highest temperature: 34.0
```

10. Conclusion

The program practices lists, loops, sum(), max(), and average calculations.

Case Study 6: Student Attendance Eligibility

1. Problem Statement

A college checks whether students meet a minimum attendance percentage to be eligible for an examination.

2. Objectives

• Input classes held and attended.

• Calculate attendance percentage.

• Display eligible or not eligible.

3. Assumptions

• Classes held must be greater than zero.

• Attendance percentage of 75% or more is eligible.

• Attended classes cannot exceed classes held.

4. Algorithm

1. Start.

2. Input classes held and classes attended.

3. Calculate attended ÷ held × 100.

4. If percentage is at least 75, display Eligible; otherwise display Not eligible.

5. Stop.

5. Flowchart



6. Pseudocode

```
START
  INPUT held, attended
  percentage = attended / held * 100
  IF percentage >= 75
      DISPLAY 'Eligible'
  ELSE
      DISPLAY 'Not eligible'
  DISPLAY percentage
STOP
```

7. Python Source Code

```
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))

if held <= 0 or attended < 0 or attended > held:
    print("Invalid attendance details")
else:
    percentage = attended / held * 100
    print("Attendance:", round(percentage, 2), "%")
    if percentage >= 75:
        print("Eligible for examination")
    else:
        print("Not eligible for examination")
```

8. Test Cases

Classes held	Attended	Expected result	
100	80	80.0	Eligible
40	25	62.5	Not eligible
20	15	75.0	Eligible

9. Sample Output

```
Enter total classes held: 100
Enter classes attended: 80
Attendance: 80.0 %
Eligible for examination
```

10. Conclusion

This example combines arithmetic, validation, and conditional statements to check a simple eligibility rule.

Case Study 7: Canteen Bill Generator

1. Problem Statement

A college canteen sells snacks at fixed prices. The program calculates the bill for selected quantities.

2. Objectives

• Store item prices in a list.

• Input quantities for three items.

• Calculate and display the total bill.

3. Assumptions

• Prices are fixed: sandwich ■40, juice ■25, and samosa ■15.

• Quantities are non-negative whole numbers.

4. Algorithm

1. Start.

2. Store item names and prices in lists.

3. Input quantity for each item using a loop.

4. Multiply each quantity by its price and add to total.

5. Display item subtotals and total bill.

6. Stop.

5. Flowchart

```
START
  |
Store item names and prices
  |
Input quantities in loop
  |
Calculate each subtotal
  |
Add subtotals
  |
Display bill
  |
 STOP
```

6. Pseudocode

```
START
  items = ['Sandwich', 'Juice', 'Samosa']
  prices = [40, 25, 15]
  total = 0
  FOR each item index
      INPUT quantity
      subtotal = quantity * price
      total = total + subtotal
      DISPLAY item, subtotal
  DISPLAY total
STOP
```

7. Python Source Code

```
items = ["Sandwich", "Juice", "Samosa"]
prices = [40, 25, 15]
total = 0

for i in range(len(items)):
    quantity = int(input("Enter quantity of " + items[i] + ": "))
    subtotal = quantity * prices[i]
    total = total + subtotal
    print(items[i], "subtotal: Rs.", subtotal)

print("Total bill: Rs.", total)
```

8. Test Cases

Example quantities	Calculation	Total
2 sandwiches, 1 juice, 3 samosas	80 + 25 + 45	150
1 sandwich, 0 juice, 0 samosas	40	40

9. Sample Output

```
Sandwich subtotal: Rs. 80
Juice subtotal: Rs. 25
Samosa subtotal: Rs. 45
Total bill: Rs. 150
```

10. Conclusion

The program uses parallel lists, a loop, multiplication, and accumulation to prepare a small bill.

Case Study 8: Library Fine Calculator

1. Problem Statement

A library charges a small fine when a book is returned late. This program calculates the fine from the number of overdue days.

2. Objectives

• Input overdue days.

• Apply a daily fine rate.

• Display the total fine.

3. Assumptions

• No fine is charged for zero or fewer overdue days.

• The fine is ■2 per day for days 1–5 and ■5 per day after day 5.

• The first five days are charged at ■2 each even when the delay is longer.

4. Algorithm

1. Start.

2. Input overdue days.

3. If days are at most zero, fine is zero.

4. If days are between one and five, fine is days × 2.

5. Otherwise, fine is 5 × 2 plus remaining days × 5.

6. Display fine and stop.

5. Flowchart

```
START
  |
Input overdue days
  |
Days <= 0?
 | Yes: fine = 0
 | No
Days <= 5?
 | Yes: fine = days * 2
 | No: fine = 10 + (days-5)*5
  |
Display fine
  |
 STOP
```

6. Pseudocode

```
START
  INPUT days
  IF days <= 0: fine = 0
  ELSE IF days <= 5: fine = days * 2
  ELSE: fine = 10 + (days - 5) * 5
  DISPLAY fine
STOP
```

7. Python Source Code

```
days = int(input("Enter overdue days: "))

if days <= 0:
    fine = 0
elif days <= 5:
    fine = days * 2
else:
    fine = 5 * 2 + (days - 5) * 5

print("Library fine: Rs.", fine)
```

8. Test Cases

Overdue days	Fine
0	0
4	8
8	25

9. Sample Output

```
Enter overdue days: 8
Library fine: Rs. 25
```

10. Conclusion

This program demonstrates multi-branch conditions and a simple progressive fine calculation.

Case Study 9: Number Guessing Game

1. Problem Statement

A simple game asks the player to guess a fixed secret number and gives feedback on whether the guess is correct, too low, or too high.

2. Objectives

• Read a guess from the player.

• Compare it with a secret number.

• Display helpful feedback.

3. Assumptions

• The secret number is 7.

• The player enters an integer.

• This beginner version allows one guess.

4. Algorithm

1. Start.

2. Set secret number to 7.

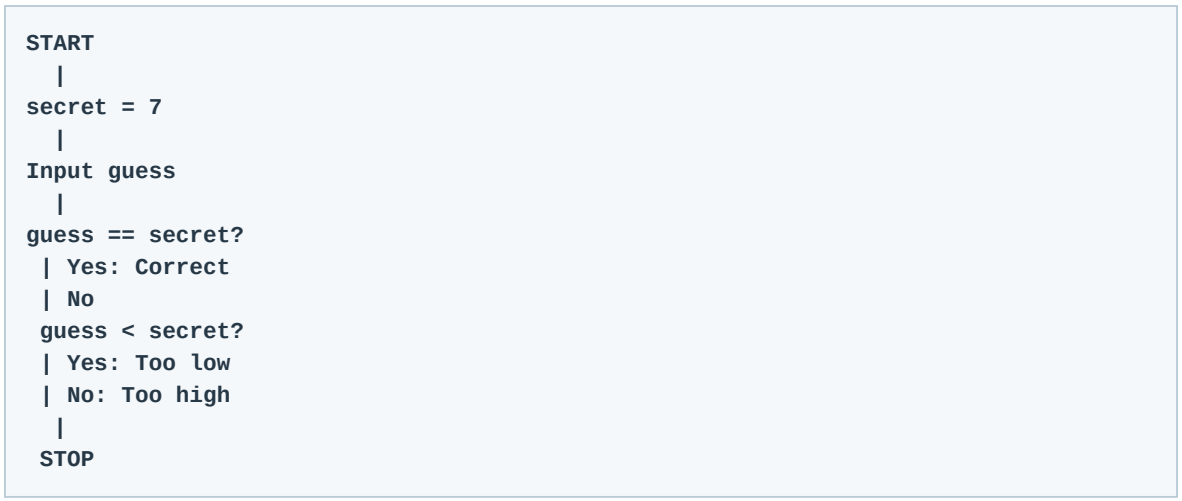
3. Input the player's guess.

4. If the guess equals the secret number, display Correct.

5. If the guess is smaller, display Too low; otherwise display Too high.

6. Stop.

5. Flowchart



6. Pseudocode

```
START
  secret = 7
  INPUT guess
  IF guess == secret
      DISPLAY 'Correct!'
  ELSE IF guess < secret
      DISPLAY 'Too low'
  ELSE
      DISPLAY 'Too high'
STOP
```

7. Python Source Code

```
secret = 7
guess = int(input("Guess the number (1 to 10): "))

if guess == secret:
    print("Correct! You guessed the number.")
elif guess < secret:
    print("Too low. Try a larger number next time.")
else:
    print("Too high. Try a smaller number next time.")
```

8. Test Cases

Guess	Expected result
7	Correct
3	Too low
9	Too high

9. Sample Output

```
Guess the number (1 to 10): 3
Too low. Try a larger number next time.
```

10. Conclusion

The game introduces comparisons and nested decision paths in a fun, easy-to-understand example.

Case Study 10: Product Stock Reorder Checker

1. Problem Statement

A small shop tracks the stock of a few products and checks which products need to be reordered.

2. Objectives

• Store product names and quantities in nested lists.

• Compare each stock quantity with a reorder level.

• Display products that need restocking.

3. Assumptions

• Each product record contains a name, current quantity, and reorder level.

• A product needs restocking when current quantity is less than or equal to its reorder level.

4. Algorithm

1. Start.

2. Create a nested list of product records.

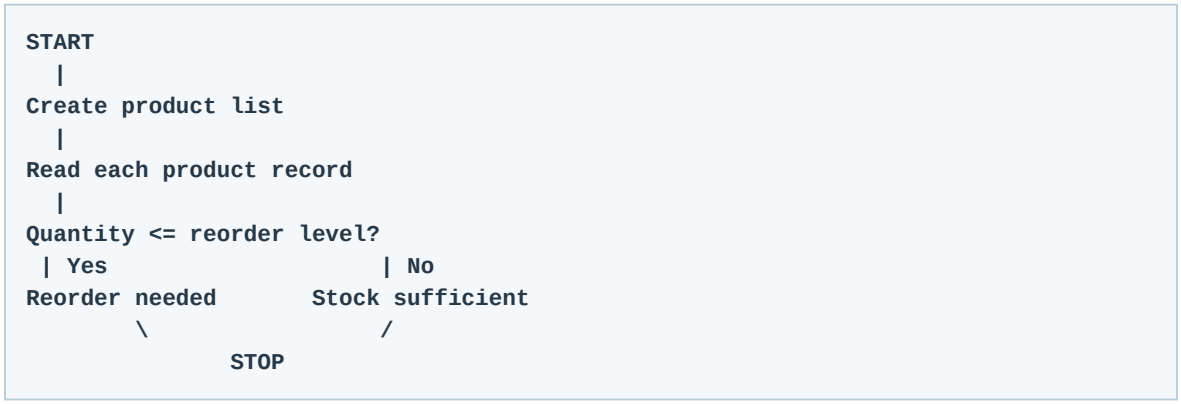
3. Loop through each product.

4. Compare current quantity with reorder level.

5. Display Reorder needed or Stock sufficient.

6. Stop.

5. Flowchart



6. Pseudocode

```
START
  products = nested list of product details
  FOR each product in products
      IF quantity <= reorder_level
          DISPLAY product, 'Reorder needed'
      ELSE
          DISPLAY product, 'Stock sufficient'
STOP
```

7. Python Source Code

```
products = [
    ["Notebook", 8, 10],
    ["Pen", 25, 12],
    ["Pencil", 5, 8],
    ["Eraser", 14, 6]
]

for product in products:
    name = product[0]
    quantity = product[1]
    reorder_level = product[2]

    if quantity <= reorder_level:
        print(name, "- Reorder needed")
    else:
        print(name, "- Stock sufficient")
```

8. Test Cases

Example	Expected result
Notebook: 8, level 10	Reorder needed
Pen: 25, level 12	Stock sufficient
Pencil: 5, level 8	Reorder needed

9. Sample Output

```
Notebook - Reorder needed
Pen - Stock sufficient
Pencil - Reorder needed
Eraser - Stock sufficient
```

10. Conclusion

This case study uses nested lists, loops, indexing, and conditions to make a basic inventory decision.